

PORTADA

Arquitectura de Software

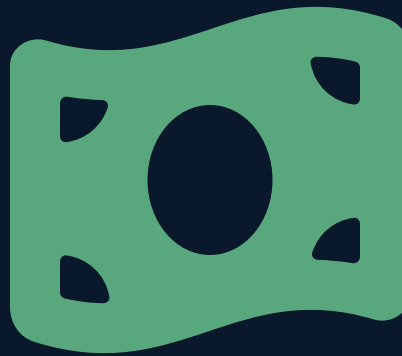
Semana 2 — Historia de la Arquitectura de Software

Universidad de Antioquia · Ingeniería de Sistemas · 2554608 · 2026-2



APERTURA

**5.000 millones de dólares
por una palabra nueva**



CASO · FICHA TÉCNICA

IBM System/360, 1964

Empresa	IBM (International Business Machines)
Producto	System/360 — familia de computadoras mainframe
Anuncio	7 de abril de 1964
Inversión	Cerca de 5.000 millones de dólares de la época — la revista <i>Fortune</i> la llamó "la apuesta de 5.000 millones de IBM"
Modelos lanzados	6 modelos distintos, de capacidad muy diferente, anunciados el mismo día
Autores del concepto	Gene Amdahl, Gerrit Blaauw y Fred Brooks
Legado	Primer uso documentado y explícito del término "arquitectura" aplicado a un sistema de cómputo

CASO · EL PROBLEMA

Antes del 360: familias incompatibles

Hasta comienzos de los años 60, cada modelo de computadora de IBM tenía su **propio diseño interno y su propio software** — un programa escrito para un modelo no corría en otro, ni siquiera dentro de la misma empresa.

- Un cliente que necesitaba más capacidad debía **reescribir todo su software** al cambiar de modelo.
- IBM mantenía en paralelo varias líneas de producto que no compartían nada entre sí.
- El costo de crecer —para el cliente y para IBM— era enorme.

CASO · LA SOLUCIÓN

Un mismo diseño lógico, muchas implementaciones

El equipo del System/360 propuso algo distinto: definir un **conjunto de instrucciones y comportamientos comunes** (lo que el programador ve) que fuera **independiente** de cómo cada modelo lo implementara por dentro (los circuitos, la velocidad, el costo).

Un mismo programa podía correr en los 6 modelos, desde el más pequeño hasta el más potente, sin reescribirse.

Para nombrar esa separación —lo que el programador ve, distinto de cómo se construye por dentro— el equipo necesitó una palabra nueva.

CASO · LA PALABRA NUEVA

Blaauw y Brooks acuñan "arquitectura"

Arquitectura es la estructura de un sistema tal como la ve el programador, es decir, la conceptualización fundamental de su comportamiento, en contraste con la organización del flujo de datos y de control, el diseño lógico y la implementación física.

Gerrit Blaauw y Fred Brooks tomaron prestado el término del mundo de la construcción —donde "arquitectura" distingue el diseño de un edificio de los materiales concretos con que se levanta— para describir exactamente la misma distinción en computadoras.

Fred Brooks es también el autor de "The Mythical Man-Month" (1975), lectura clásica de ingeniería de software.

TRANSICIÓN

De un mainframe a todo el software

El System/360 acuñó la palabra pensando en **hardware** — separar "qué hace la máquina" de "cómo está construida por dentro". Con los años, esa misma idea se trasladó al software: separar **qué hace un sistema y cómo están organizadas sus partes**, de los detalles de cada línea de código.

Para entender por qué la arquitectura de software se piensa hoy como se piensa, hay que recorrer las eras que siguieron a 1964.

CONCEPTO

Línea de tiempo: cinco eras

Era	Década aproximada	Rasgo arquitectónico dominante
Mainframe	1960s–1970s	Un único sistema centralizado, procesamiento por lotes
Minicomputadoras y Unix	1970s–1980s	Sistemas más pequeños, terminales conectadas, tiempo compartido
Cliente/servidor	1980s–1990s	Trabajo dividido entre PC (cliente) y servidor central
Web	1990s–2000s	El navegador como cliente universal, servidores n-capas
Cloud y microservicios	2010s–hoy	Servicios distribuidos, elásticos, desplegados en la nube



CONCEPTO · ERA MAINFRAME

Todo en una sola máquina central

El System/360 y sus sucesores definieron esta era: **una computadora central** —cara, grande, compartida por toda una organización— a la que los usuarios acceden mediante **terminales "tontas"** que solo envían pantallas de texto, sin capacidad propia de procesamiento.

- El procesamiento se hacía por **lotes (batch)**: se programaba de noche, se revisaba de día.
- Toda la lógica, los datos y la interfaz vivían en el mismo lugar físico.



CONTRA EJEMPLO

Mito: "el mainframe era una mala arquitectura"

Es tentador ver la era del mainframe como "primitiva" comparada con hoy. **No lo era para su contexto.**

- El hardware era extremadamente caro: centralizar todo en una sola máquina compartida era la opción económicamente racional.
- El procesamiento por lotes era eficiente para el tipo de carga de trabajo de la época (nómina, contabilidad, inventarios).
- Muchos sistemas mainframe de esa arquitectura **siguen en producción hoy** en bancos y aseguradoras, por su estabilidad probada.

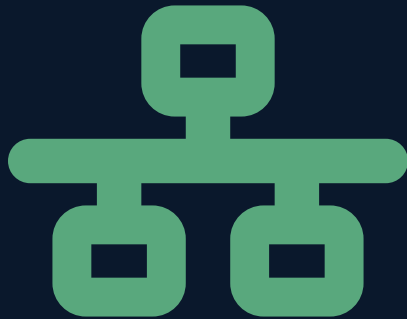
Una arquitectura no es "buena" o "mala" en abstracto — es apropiada o no para su contexto y sus restricciones.

CONCEPTO · ERA CLIENTE/SERVIDOR

El trabajo se reparte

Con la llegada de la PC (años 80) y las redes locales, apareció una nueva división del trabajo: el **cliente** (una PC con interfaz gráfica propia) y el **servidor** (una máquina que centraliza los datos y algunas reglas).

Modelo	Cómo se divide el trabajo
2 capas	Cliente con interfaz + lógica; servidor solo con la base de datos
3 capas	Cliente (interfaz), servidor de aplicación (lógica), servidor de datos (base de datos)



CONCEPTO · ERA WEB

El navegador como cliente universal

Con la Web (años 90–2000) el cliente dejó de ser una aplicación instalada por cada usuario y pasó a ser **el navegador**: un mismo cliente genérico capaz de mostrar cualquier aplicación, sin instalación previa.

- El servidor pasó a organizarse en **n capas**: presentación, lógica de negocio, acceso a datos.
- El protocolo **HTTP** se volvió el conector universal entre cliente y servidor (visto la semana pasada).
- Nacieron los primeros frameworks web con el fin de estandarizar esa organización.

CONCEPTO · ERA CLOUD Y MICROSERVICIOS

Sistemas que se despliegan, no que se instalan

Desde la década de 2010, la infraestructura misma se volvió parte de la arquitectura: en vez de comprar y mantener servidores propios, los sistemas se despliegan sobre **infraestructura en la nube**, dividida en **servicios pequeños e independientes** (microservicios) que escalan por separado según la demanda.

Este patrón —y sus alternativas (SOA, monolito modular, serverless)— se estudia en detalle en la Unidad 2 del curso.



TENSIÓN CONCEPTUAL

¿Cada era "reemplaza" a la anterior?

Es tentador narrar la historia como una escalera donde cada era supera a la anterior. **La realidad es más desordenada:**

- Bancos y aerolíneas siguen operando mainframes de los años 70, ahora conectados a APIs web modernas.
- La arquitectura cliente/servidor de 2 capas sigue siendo la elección correcta para muchas aplicaciones internas pequeñas.
- Microservicios no es "más moderno y por tanto mejor": introduce complejidad que no todo proyecto necesita.

¿Tiene sentido, entonces, hablar de "progreso" en arquitectura de software?



INTERACCIÓN

Ubiquen la era

Para cada sistema, identifiquen a qué era pertenece su arquitectura dominante y por qué:

1. El sistema de un banco que procesa la nómina de una empresa una vez al mes, de noche.
2. Una aplicación de escritorio instalada en cada PC de una oficina, que se conecta a un servidor de base de datos central.
3. Una plataforma de streaming que despliega decenas de servicios independientes en la nube, cada uno escalando según la demanda.
4. Un portal universitario al que se accede desde cualquier navegador, sin instalar nada.

5 minutos · respondan en voz alta o en el chat

CONCEPTO

Por qué conocer esta historia importa

Los patrones no desaparecen: **se reinterpretan**. La arquitectura cliente/servidor de los 80 reaparece hoy como "frontend + backend" (visto la semana pasada). El procesamiento por lotes del mainframe reaparece en los *batch jobs* nocturnos de sistemas modernos.

Entender de dónde viene cada patrón ayuda a reconocer cuándo conviene aplicarlo de nuevo — no como nostalgia, sino como herramienta de diseño.

SÍNTESIS

Ideas clave de hoy

1. El término "**arquitectura**" aplicado a computación nace en 1964, con el IBM System/360, para separar "qué hace el sistema" de "cómo está construido por dentro".
2. La historia recorre cinco eras: **mainframe, minicomputadoras/Unix, cliente/servidor, web, cloud/microservicios**.
3. Ninguna era "reemplaza" completamente a la anterior — muchos patrones antiguos siguen vigentes en contextos apropiados.
4. Una arquitectura es apropiada o no **según su contexto**, no "buena" o "mala" en abstracto.
5. Los patrones se reinterpretan: el cliente/servidor de los 80 es, en esencia, el frontend/backend de hoy.

INSTRUCCIONES

Taller de hoy — Implementación de ejemplo

Demostración guiada por el docente — 30 a 40 minutos

El docente muestra en vivo **dos versiones del mismo ejemplo** (el backend en Spring Boot de la semana pasada) para hacer tangible el salto de era cliente/servidor a arquitectura web moderna:

- **Versión "2 capas"**: el backend accede directamente a una lista en memoria, sin separación de capas.
- **Versión "n capas"**: el mismo backend, ahora separado en controlador, servicio y repositorio.

De 2 capas a n capas, en el mismo código

```
// Versión "2 capas": todo junto en el controlador
@RestController
public class SaludoController {
    private List<String> mensajes = new ArrayList<>();

    @GetMapping("/api/saludo")
    public String obtener() {
        return "Hola desde el backend"; // lógica y datos mezclados
    }
}

// Versión "n capas": responsabilidades separadas
@RestController
class SaludoController {
    @Autowired SaludoService servicio;
    @GetMapping("/api/saludo")
    public String obtener() { return servicio.generar(); }
}
class SaludoService {
    @Autowired SaludoRepository repo;
    public String generar() { return repo.buscarMensaje(); }
}
```

No hay entregable de estudiantes: el objetivo es reconocer visualmente la diferencia entre una arquitectura de 2 capas y una de n capas.

CIERRE

Para la próxima sesión

- Pensar en un framework o tecnología que usen hoy y preguntarse: ¿qué problema de una era anterior vino a resolver?
- Traer identificado, de su propio proyecto de Fábrica Escuela, en qué "era" arquitectónica se ubica su stack actual.

El jueves continuamos con la evolución de la arquitectura de software: cómo cambiaron las tecnologías y los frameworks a lo largo de estas mismas eras.

